

TF PROTOCOL

ver 2.5.2

**“To my wife Yanin for her infinite patience
for my time consuming projects.”**

Overview

The TF Protocol works by sending text commands from client to the server in a TCP connection. Every time a command is received at the server it responds with a command execution status. The buffer used for all the communication must be of $512 * 1024$ bytes.

The communication between the server and the client occurs in an encrypted way. This encryption is a client-to-server one. It uses at first public key encryption to exchange a session key. This session key is used for the encryption to transmit and receive commands, data, etc.

This protocol has two works modes. The standard mode, and the notification mode. In the standard mode the server holds on for commands from the client, then it executes it, and it returns the result and the possible extra data depending on the command. In the notification mode the client holds on for the server to receive data.

Every sent or received data must be preceded by a header indicating the size of the message that will be received. The only exception of this, are for those modules and commands that implements their own header mechanism. The header for the current version of TF PROTOCOL looks like this:

```
struct tfhdr {  
    int32_t sz;  
}
```

It only contains one field of 32bits (integer). Before sending it must be converted to network byte order (big-endian). It must be two of

this header to allow multi-threaded communication (full-duplex comm). One used for TX 'transmit data' and the other one used for RX 'receive data'.

Starting communication

The server has the following internal information structure.

```
/* Protocol version length */
#define PROTOLEN 32
/* Hash application identity length. */
#define HASHLEN 256

struct tfproto {
    /* Protocol version */
    char proto[PROTOLEN];
    /* Hash identity */
    char hash[HASHLEN];
    /* Directory for the database. */
    char dbdir[PATH_MAX];
    /* Number of port for comm. */
    char port[PORTLEN];
}
```

The **proto** member holds the protocol version. The **hash** member holds a message digest that identifies the client in order to allow the use of the protocol. The **dbdir** member holds the directory for the data that clients require to store. The **port** member indicates the number of ports in which the server will listen for connections. As stated before the **hash** has to be less than 256 bytes long; and the **proto** less than 32 bytes long.

In order to match this structure when a client starts a connection, there are some steps that must be followed.

Beginning the connection

The client starts the connection by sending the version of the protocol that it expects to use. The exact string of this depends of the configuration of the protocol. There is no restriction to numbers.

Once the server validates the protocol version it responds with a command status or an error condition.

OK for approved.

FAILED 2 : Incompatible protocol.

If the server returns with a failure status it has three parts. The string FAILED which indicates an error. The number after one space character could be used to identify the error with integer type so there is no need to parse the following string. The next string after the number is a text that describes, mostly for debug purpose, the nature of the error.

This dynamic of sending commands from client and once the server processes it, it returns an exit status, it repeats for every command of the protocol. There is one exception, some command may not indicate an error code and the string indicating the error.

After the server validates the protocol version and sends back to the client the OK command, it goes to wait again, it is time for the public key of the client to start the encryption system. If the protocol validation fails the connection is terminated by the server.

The encryption system

The TF PROTOCOL transmits and receives data encrypted. The client must have the public key of the server. The RSA must be 2048 bit and RSA_PKCS1_OAEP_PADDING padding. The procedure of exchanging keys and generating session keys occurs as follows:

- 1) The client generates an arbitrary session key which max length cannot exceed $2048 / 8 - 42$ bytes, and must be at least 16 bytes.
- 2) The client encrypts the session key with the server's public key and sends it.
- 3) The server decrypts the session key with its own private key.
- 4) The server sends back OK if the process has been successful, otherwise sends back FAILED, yet in plain text.

If there is an error in this process the connection is terminated by the server.

The encryption session key is an array of at least 16 bytes up to $2048 / 8 - 42$ that is generated in a random way and it will be xored with the data sent or received from the server. As soon as the server and the client are in possession of the session key, it must be extracted from it a **seed**. The seed are the first 8 bytes as a 64bit integer.

The algorithm to encrypt and decrypt data with the session key in C looks like this one.

```
#define KEYLEN 32
char rndkey[KEYLEN] = "the encryption key";
char data[] = "the data";
```



```

int len = sizeof data;
int decrypt;
int c = 0, keyc = 0;
for (; c < len; c++, keyc++) {
    if (keyc == KEYLEN)
        keyc = 0;
    if (decrypt) {
        /* Unpack received data */
        data[c] = data[c] - (seed >> 56 & 0xFF);
        /* Decrypt received data */
        data[c] ^= rndkey[keyc];
    } else if (encrypt) {
        /* Encrypt data to send */
        data[c] ^= rndkey[keyc];
        /* Pack data to send */
        data[c] = (seed >> 56 & 0xFF) + data[c];
    }
    /* Change seed and encryption key */
    seed = seed * (seed >> 8 & 0xFFFFFFFF) +
        (seed >> 40 & 0xFFFF);
    if (seed == 0)
        seed = *(int64_t *) rndkey;
    rndkey[keyc] = seed % 256;
}

```

From this point forward, all the communication will be encrypted and decrypted with the session key through an algorithm like the above. As the session key changes over time, it must be a copy of those two factors: **session key** and **seed**. This allows in a multi-thread environment perform reading and writing operations simultaneously without interference to each other.

It is also required to preserve the session key in its original form, this is, without any transformation. This is a provision in case any extended subsystem requires the original key.

The next step is to send to the server the hash identity. It responds as usual with OK or FAILED exit status.

The summary.

- 1) The client starts a connection.
 - 2) The client sends the protocol version.
 - 3) The server responds one of these:
 - OK
 - FAILED 2 : Incompatible protocol.
 - 4) The client sends the session key encrypted with the server's public key.
 - 5) The server decrypts the session key which will be used to encrypt the rest of the communication. The response is one of these:
 - OK
 - FAILED 25 : Bad public rsa encryption key.
- From this point on, encrypted.
- 6) The client sends the hash.
 - 7) The server validates it and responds one of these:
 - OK
 - FAILED 3 : Invalid hash string.

Once all the above steps are passed successfully, the communication begins with the command interface. In case of a failure the communication is terminated by the server.

Security enhancement

This security enhancement provides a real user 'system user' mechanism to enforce through the file system driver the directory and file access permission. It works in one of the following ways:

Single-user:

The TFProtocol deployment creates a default user for the protocol daemon. The daemon works on behalf of this user and there is no need for further settings, and login is not required. In this case every connection to the protocol will work with that user permission.

Multi-user:

The TFProtocol does not define any default user; instead, it defines a set of user/password pairs. In this case, once the connection is established, the client should call as a first command 'LOGIN' to login at the server. If the user/password combination is a valid one, the server will use that user as the persona of the process. The fundamental difference in regards to the 'Single-user' mentioned above is that every connection can login with a different user and thus its respective access permission.

This way every client using the protocol can create directories and files with ownership and security access enforced by the file-system driver. This allows clients, if used properly, to have directories and files that no one else can read/write.

Only the following commands can be called before **LOGIN**:

END, **KEEPALIVE**, and **PROCKEY**

In order to allow this security enhancement to work, the protocol defines a few special commands for this task: **LOGIN**, **CHMOD**, **CHOWN**. You can find more information in the next section.

In-jail:

The TFProtocol deployment sets whether in-jailing will be required or not. The term in-jail refers to restrict the access of the TFProtocol daemon to a particular directory. If in-jailing is required, it has to be called after a possible login requirement, but before every other command. For further information see the **INJAIL** command documentation.

Only the following commands can be called before **INJAIL** if multi-user is set:

END, **LOGIN**, **KEEPALIVE**, and **PROCKEY**

If single-user is set, only the **END** command can be called before **INJAIL**.

Only the following commands can be called before **INJAIL** if multi-user is set:

TFProtocol Secure Filesystem

The TFProtocol Secure Filesystem, now on: TFPSFS, is intended to provide a way to set permissions for a directory based on defined identities and their assigned permissions. It works as follow:

A directory that belongs to this domain will contain inside of it three files: .securefs_metadata, .securefs_metadata.rd, .securefs_metadata.lck. With those three files the system guarantee the usage of TFPSFS.

The commands that works with this domain has a tag indicated their support. If a command is executed on a directory that do not has those files, the command is executed as always. If a command that support TFPSFS and the directory in which is executed belongs to the that domain -contains the 3 files-, the command will check for permission as follow: It will check it's identity set by SETFSID command against the directory permissions. If the identity exist, but has not enough permissions, the command will fail. If the identity does not exist and -everybody else- identity exist for that directory, it will be used. If the -everybody else- identity has not enough permissions for the operation, the command will fail. If neither the identity set by SETFSID or the -everybody else- identity exists, the command will fail. If the command does not support TFPSFS and the directory in which is executed belongs to TFPSFS, the command will fail.

The security permission is a 32bit unsigned integer bitwise OR mask of the following values:

```
/* Set identity and it's permissions for the directory. */  
SECFS_SETPERM = 1
```

To use it, requires that the process identity set by SETFSID has this permission to operate in the directory, or be the first time SETFSPERM is called for the directory. Unlike other commands, if the identity set by SETFSID does not have it, it will not fall through to 'everybody else' identity.

```
/* Remove identity and it's permissions for the directory. */
```

```
SECFS_REMPERM = 2
```

To use it, requires that the process identity set by SETFSID has this permission to operate in the directory. Unlike other commands, if the identity set by SETFSID does not have it, it will not fall through to 'everybody else' identity.

```
/* Set Read permission for files in the directory. */
```

```
SECFS_RFILE = 4
```

```
/* Set Write -Create, Truncate, etc.- permission for files in the directory. */
```

```
SECFS_WFILE = 8
```

```
/* Set List permission for files in the directory. */
```

```
SECFS_LDIR = 16
```

```
/* Set Remove Directory permission for directories in the directory. */
```

```
SECFS_RMDIR = 32
```

```
/* Set Make Directory permission for directories in the directory. */
```

```
SECFS_MKDIR = 64
```

```
/* Set Delete permission for files in the directory. */  
SECFS_DFILE = 128
```

```
/* Set STAT permission for files and directories in the directory. */  
SECFS_STAT = 256
```

```
/* Set File Updating Timestamps permission for files and directories  
in the directory. */  
SECFS_FDUPD = 512
```

```
/* Set UNIX permission changing for files and directories in the  
directory. */  
SECFS_UXPERM = 1024
```

```
/* Set List Directory Recursively permission for files and directories  
in the directory. */  
SECFS_LRDIR = 2048
```

Whenever this mask is used it must be converted to its string base10 representation. For example, to use it in the SETFSPERM command. In the future many other permissions could be added to those above indicated.

The -everybody else- identity is set by calling the SETFSPERM only with the mask, leaving in blank the identity part:

```
SETFSPERM :128 /path/to/dir.
```

It can be removed by calling REMFSPERM leaving the identity part in blank:

REMFSPERM 'white-space here' /path/to/dir.

The -everybody else- identity, as stated before, is used to provide a last-resource permission for a process. This requires that the directory has the -everybody else- identity set. It cannot be used with the SETFSPERM and REMFSPERM commands even if those permissions are set for the -everybody else- identity.

A note of caution:

Be aware that TFProtocol can only enforce the security provided by TFPSFS in the Main Command Interface. The extended subsystems are not compelled to enforce it. If some subsystem is going to be used along with the TFPSFS feature, the subsystem documentation must be carefully read.

The command interface

The server can receive the following commands. Every command is formed by a name and optional arguments. The double quote of the command arguments in the examples, as in the return status examples, are not part of the command syntax, it is just for reading convenience.

UNKNOWN

This is the response sent by the server if the command is not recognized.

END

END command is sent by the client to the server in order to terminate the TCP connection.

ECHO “example”

ECHO command is for debug propose. Once sent the server replays back the exact same thing the client sent, including the ECHO word.

[Supports TFPSFS]

MKDIR “path/to/new/dir”

MKDIR command creates a directory at the specified path. The return status of this command could be:

OK

FAILED 4 : Directory already exist.

FAILED 1 : Access denied to location.

FAILED 30 : Directory is locked.

If the parameter is left empty the daemon protocol directory will be used. So it responds the FAILED 4 return status.

Secured Directory

This feature allows the creation of a kind of directory that cannot be listed with any command. In order to access it, the name must be known in advance. The directory name may contain any character, except the decimal zero “0” and the path separator “/”, and must end with an “.sd” extension. The length of the directory name must not exceed 255 characters including the extension.

In order to create strong secured directories, the following technique could be used: Combine a user’s name with a very strong password and apply to them the SHA256 or any other secure hash function. The resulting hash must be scanned to remove or change from it any occurrence of the decimal zero ‘0’ -the null character- or the path separator ‘/’. If the hash is converted to hexadecimal format -which is usual- , it is guaranteed that the resulting string will contain only numbers and letters. If more security is needed, this process can be repeated recursively.

[Supports TFPSFS]

DEL “/path/to/file/to/del”

DEL command deletes the specified file. This is a special command because is the only one who can operate in a locked directory. If it is

the case, then DEL can only delete exactly one file: The locking file.
The return status of this command could be:

OK

FAILED 1 : Access denied to location.

FAILED 9 : File does not exist.

FAILED 8 : Requested file is a directory.

FAILED 30 : Directory is locked.

FAILED

If used without an argument the protocol daemon directory is used.
So it return the FAILED 8 return status.

[Supports TFPFS]

RMDIR “/path/to/dir/to/remove”

RMDIR removes a specified directory recursively. The return status of this command could be:

OK

FAILED 1 : Access denied to location.

FAILED 30 : Directory is locked.

FAILED 15 : Directory to remove still exist.

If “/path/to/dir/to/remove” has secure directories inside of it, they will be not removed.

If used without an argument the protocol daemon directory is used.
So it returns the FAILED 15 return status due the impossibility to delete the daemon directory. However, every other directory will be deleted.

[Supports TFPFS]

COPY “/path/to/source/file” | “/path/to/new/file”

COPY copy the file indicated by the first parameter of the command to the file indicated by the second one. The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 19 : Directory can't be linked.

FAILED 12 : File already exist.

FAILED 17 : Source file does not exist.

FAILED 18 : Copy general error.

FAILED 30 : Directory is locked.

[Supports TFPFS]

TOUCH “path/to/file/to/create”

TOUCH creates a new file in the specified directory. The return status of this command could be:

OK

FAILED 1 : Access denied to location.

FAILED 12 : File already exist.

FAILED 24 : Error creating new file.

FAILED 30 : Directory is locked.

If used without an argument the protocol daemon directory is used. So it return the FAILED 12 return status.

DATE

DATE returns the number of elapsed seconds since the epoch, and arbitrary point in the time continuum, which is the Gregorian calendar time Jan 1 1970 00:00 UTC. The return status of this command could be:

OK “timestamp”

UPDATE

UPDATE returns the number of elapsed seconds and microseconds since the epoch -separated by dot-, and arbitrary point in the time continuum, which is the Gregorian calendar time Jan 1 1970 00:00 UTC. The return status of this command could be:

OK “seconds.microseconds”

NDATE

NDATE returns the number of elapsed seconds and nanoseconds since the epoch -separated by dot-, and arbitrary point in the time continuum, which is the Gregorian calendar time Jan 1 1970 00:00 UTC. The return status of this command could be:

OK “seconds.nanoseconds”

DATEF

DATEF returns the current date of the server in human readable format “yyyy-mm-dd HH:MM:SS” UTC. The return status of this command could be:

OK “yyyy-mm-dd HH:MM:SS”

DTOF “timestamp”

DTOF converts date in Unix timestamp format -seconds since the epoch- in human readable format “yyyy-mm-dd HH:MM:SS” UTC. The return status of this command could be:

OK “yyyy-mm-dd HH:MM:SS”

FAILED 16 : Missing parameter from command.

FAILED 26 : Date is not representable.

FTOD “ yyyy-mm-dd HH:MM:SS”

FTOD converts date in human readable format “ yyyy-mm-dd HH:MM:SS” to its Unix timestamp format. The return status of this command could be:

OK “timestamp”

FAILED 16 : Missing parameter from command.

[Supports TFPFS]

FSTAT “/path/to/file/or/directory”

FSTAT returns statistics of a file or directory in the form "D | F | U FILE-SIZE LAST-ACCESS LAST-MODIFICATION" where D stands for directory; F stands for file; and U stands for unknown, only one of them is reported. The “FILE-SIZE” is reported in bytes in a integer of 64 bits and the number could be up to 20 digits long. The “LAST-ACCESS” and “LAST-MODIFICATION” are both timestamps. The return status of this command could be:

OK “D | F | U FILE-SIZE LAST-ACCESS LAST-MODIFICATION”

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 9 : File does not exist.

FAILED 30 : Directory is locked.

[Supports TFPSFS]

FUPD “/path/to/file/or/directory”

FUPD update the timestamps of the file or directory to server current time. The return status could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 9 : File does not exist.

FAILED 27 : Timestamp updating failed.

FAILED 30 : Directory is locked.

[Supports TFPSFS]

CPDIR “/path/to/source/directory” | “/path/to/destination/directory”

CPDIR copy recursively the source directory into a new created directory specified in the second parameter of the command. The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 20 : Source path is not a directory.

FAILED 4 : Directory already exist.
FAILED 21 : Error replicating directory tree.
FAILED 30 : Directory is locked.

XCOPY “newname” “/path/to/source/file” | “pattern”

XCOPY copies the source file specified in the second parameter into every directory in the tree that matches “pattern” with the name “newname” specified in the first parameter.

This is a best-effort command instead of super-reliable one. If any one of the destinations are locked or the file exists instead of return a failed status code, XCOPY skips it silently. The return status of this command are relative to the source file/directory.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.
FAILED 1 : Access denied to location.
FAILED 17 : Source file does not exist.
FAILED 23 : Source path is not a file.
FAILED 30 : Directory is locked.

XDEL “/path/to/directory/” “filename”

XDEL deletes all files that match “filename” in the specified path at first parameter and it does recursively starting at the specified directory.

This is a best-effort command instead of super-reliable one. If any one of the path are locked or even an error occurs, instead of return a failure return status, XDEL skips it silently. The return status of this command are relative to the path specified.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 20 : Source path is not a directory.

FAILED 30 : Directory is locked.

XRMDIR “/path/to/directory/” “directory-name”

XRMDIR deletes all directories recursively that match “directory-name” starting at the specified path at first parameter.

This is a best-effort command instead of super-reliable one. If any one of the path are locked or even an error occurs, instead of return a failure return status, XRMDIR skips it silently. The return status of this command are relative to the path specified.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 10 : Directory does not exist.

FAILED 20 : Source path is not a directory.

FAILED 30 : Directory is locked.

XCPDIR “new-directory-name” “/path/to/source/directory/” |
“destination-directory-pattern”

XCPDIR copy recursively source directory specified in the second parameter to every directory in the tree that matches “destination-directory-pattern” with the name “new-directory-name” specified at first parameter.

This is a best-effort command instead of super-reliable one. If any one of the destinations are locked or the directory or file exists, instead of return a failure return code, XCPDIR skips it silently. The return status of this command are relative to the source file/directory
The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 20 : Source path is not a directory.

FAILED 29 : Source path does not exist.

FAILED 30 : Directory is locked.

LOCK “lock-filename”

LOCK specifies a filename that when it exists in a directory no operation can be done on it, except to delete the lock file. This allows a temporary blocking of a directory for any sort of operations except the one eliminating the locking file which is the command **DEL** with the path to the locking file. The above statement does not apply for commands that do not specify “FAILED 30” as possible return status. The return status of this command could be:

OK

Specify LOCK with an empty string “” or without argument is the way to turn off the locking mechanism regardless of how many locking files are installed in the directories. The notification system is not affected by the locking mechanism.

This is not a synchronization mechanism. A synchronization mechanism eliminates race conditions, LOCK does not. Don't try to

use it as such, there is a lot of windows in between. Even it is possible that some operations are completed successfully in the directory by other clients after it has been locked. The only way the client realizes of this is by comparing the files and directories timestamps against the lock file timestamp. If the lock file and any other file or directory in the locked directory has the same timestamp the client can decide whether to accept that file or directory as valid one.

[Supports TFPSFS]

SNDFILE “0 | 1” “/path/to/filename”

SNDFILE sends a file to the server. The first parameter could be either “0” for false or “1” for true. If “/path/to/filename” it exists in the server, this flag indicates whether should be overwritten or not. Once the transmission loop starts there are three keywords to continue or stop the flow: **CONT BREAK** and **OK**. The CONT keyword followed by a whitespace “ ” and the payload tells to the server that the chunk of data is part of file. The BREAK keyword tells to the server to abort the operation and it deletes the already stored data. The OK keyword tells to the server that there is no more data to be send. Every time the server receives a chunk of data it responds with the flow control keyword CONT or and status returns in case of a failure. When the SNDFILE command is sent the server responds with CONT flow control to signal that is ready for receiving the file.

The return status of this command could be:

CONT “To indicate the willingness to continue receiving the file”

OK “In response to the OK sent by the client signaling the EOF -end of file- condition or the BREAK to tell the server to stop receiving”

FAILED 13 : Quota exceeded.
FAILED 16 : Missing parameter from command.
FAILED 1 : Access denied to location.
FAILED 12 : File already exist.
FAILED 8 : Requested file is a directory.
FAILED 7 : File transfer API error.
FAILED 30 : Directory is locked.

As an illustration of the dynamic of this command we can use this example:

CLIENT SEND: SNDFILE 0 /path/to/new/file

SERVER RESPONDS: CONT

CLIENT SEND: CONT PAYLOAD-OF-DATA

“This process repeats until the server returns and failure status or the client sends BREAK to stop the flow or OK to indicate that the operation is done. In this example the first parameter which is 0 tells to the server that if the /path/to/new/file exists must not be overwritten”. The maximum number of bytes for payload is BUFSZ - CONT word and the whitespace. If buffer size is 512, then the payload cannot exceed 507 bytes. As benchmarks have been made we suggest that this command should be used only to transfer small files ‘100 MB or less’. In a1Gbps network testing the operation is completed in eleven seconds for a 100 MB file.

[Supports TFPSFS]

RCVFILE “0 | 1” “/path/to/filename”

RCVFILE receives a file from the server. The first parameter could be either “0” for false or “1” for true and tells the server whether the file must be deleted after successfully received by the client. Once the transmission loop starts there are three keywords to continue or stop the flow: **CONT BREAK** and **OK**. The CONT keyword followed by a whitespace “ ” and the payload tells to the client that the chunk of data is part of file. The BREAK keyword tells to the server to abort the operation. The OK keyword tells to the client that there is no more data to receive. Every time the client receives a chunk of data it responds with the flow control keyword CONT or BREAK to stop the operation. When the RCVFILE command is sent the server responds with CONT flow control and the payload or a return status for failure.

The return status of this command could be:

CONT PAYLOAD-OF-DATA

OK “In response to the BREAK sent by the client or to signal the EOF -end of file- condition indicating that the operation is done.”

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 9 : File does not exist.

FAILED 8 : Requested file is a directory.

FAILED 7 : File transfer API error.

FAILED 30 : Directory is locked.

As an illustration of the dynamic of this command we can use this example:

CLIENT SEND: RCVFILE 0 /path/to/filename

SERVER RESPONDS: CONT PAYLOAD-OF-DATA
CLIENT SEND: CONT

“This process repeats until the server returns OK for -end of file-condition or a failure status, or the client sends BREAK to stop the flow. In this example the first parameter which is 0 tells to the server that /path//to/filename must not be deleted after the operation is done”. The maximum number of bytes for payload is BUFSZ - CONT word and the whitespace. If buffer size is 512, then the payload cannot exceed 507 bytes. As benchmarks have been made we suggest that this command be used only to transfer small files ‘100 MB or less’. In a 1Gbps network testing the operation is completed in eleven seconds for a 100 MB file.

[Supports TFPSFS]

LS “/path/to/directory/to/list”

LS command list the directory entries for the indicated path, if the argument is missing, it lists the root directory of the protocol daemon. The return value of this command is a file with the listed content. In fact it is like issuing the command RCVFILE to a temporary file with the listed content of the directory. The file returned by LS has the following syntax.

F | D | U /path/to/file-or-directory

The paths are relative -“path/to/directory/to/list” is omitted-.

The F stands for “file”; the D for “directory” and the U for “unknown”. The return status of this command beside those for RCVFILE because RCVFILE works on behalf LS could be:

FAILED 1 : Access denied to location.
FAILED 30 : Directory is locked.
FAILED 10 : Directory does not exist.
FAILED 11 : Failed to make a temporary file.

[Supports TFPSFS]

LSR “/path/to/directory/to/list”

This command is exactly like LS but it lists recursively the specified directory.

[Supports TFPSFS]

RENAM “path/to/oldname” | “path/to/newname”

RENAM renames the file or directory specified at first parameter into the name specified at second parameter. RENAM operates atomically; there is no instant at which “newname” is non-existent between the operation’s steps if “newname” already exists. If a system crash occurs, it is possible for both names “oldname” and “newname” to still exist, but “newname” will be intact.

RENAM has some restrictions to operate.

- 1) “oldname” it must exist.
- 2) If “newname” is a directory must be empty.
- 3) If “oldname” is a directory then “newname” must not exist or it has to be an empty directory.
- 4) The “newname” must not specify a subdirectory of the directory “oldname” which is being renamed.

KEEPALIVE “1|0” “idle-time” | “interval” | “count”

KEEPALIVE sets the configuration parameters for the TCP keepalive feature. This is especially useful for clients behind NAT boxes. If there is some idle time in the established connection -no data transmission- the NAT box could close or unset the connection without the peers knowing it. In contexts where it is predictable that an established connection could be 'in silent' for long periods of time, and it is possible that clients are behind NAT boxes, it is necessary to set the TCP keepalive packets.

The first parameter of the command could be 0 or 1, meaning on or off.

The second parameter is the time (in seconds) the connection needs to remain idle before TCP starts sending keepalive probes.

The third parameter is the time (in seconds) between individual keepalive probes.

The fourth parameter is the maximum number of keepalive probes TCP should send before dropping the connection.

The return status of this command could be:

OK

FAILED 32 : Failed to set TCP keepalive parameters.

Whether the server implements a default timeout of keepalive to declare a connection dead, it is up to the implementation. In order to be sure that keepalive feature is turned off or on, it must be done explicitly.

There are two more commands ADDNTFY and STARTNTFY. We will see them in the next section.

PROCKEY

PROCKEY retrieves a unique key generated by the server's instance that communicates with the client. This unique key could be used later to identify that instance. One of these uses, but not the only one, is to test whether the server or even the socket communication line is still opened, in other words: the keepalive mechanism from the client side perspective.

The return status of this command could be:

OK

FAILED 33 : Process without unique key.

FREESP

FREESP retrieves the available space in the partition where the protocol folder is located. This command should be used only after a failure to find out if such a failure is due to the lack of space. It should not be used to guess how much space remains, and then proceed with a writing operation. The described scenario could potentially lead to a race condition. If both clients at the same time retrieve the free space and then write according to that value, one of them will fail.

The return status of this command could be:

OK "free-space"

"free-space" will be a 64bit signed integer indicating how much space left -in bytes- in the protocol partition directory at the time of the command execution.

[Supports TFPSSFS]
PUT

```
struct hpfile {  
    uint64_t offst;  
    int64_t bufsz;  
} ;
```

PUT “/path/to/filename” hpfile

PUT is part of the High Performance File Interface. It can be used to transfer files from the client to the server.

The first parameter is the file to be written, it could exist or not. If the file does not already exist, then it is created. If the file already exists, the command will overwrite it.

The second parameter goes after the “/path/to/filename” and a whitespace. It consists of two 64bit integers normalized ‘converted to network byte-order’. The first one should be unsigned and the second one, signed. The first one must range from 0 to $2^{64}-1$. The second one must range from $-(2^{63})$ to $2^{63}-1$.

The “offst” integer -the first one- is the desired offset in which the data should be written in the opened file, being a new file or an existing one.

The “bufsz” integer -the second one- is the proposed buffer size by the client.

Once the command is constructed like above, it can be sent to the server. On the server side, the protocol will evaluate the parameters and will return one of the following conditions:

OK “definitive-buffer”

FAILED 1 : Access denied to location.

FAILED 16 : Missing parameter from command.

FAILED 34 : H-P interface failed to open file descriptor.

FAILED 35 : H-P interface failed to set file position.

FAILED 36 : H-P interface failed to allocate the buffer.

“definitive-buffer” is a 64bit signed integer normalized -converted to network byte-order- with the buffer that the server finally decided to use. The server tries to create a buffer with the size indicated by the client. If that request cannot be satisfied, the server will create a buffer according to implementation-dependent criteria. In any case, if the server replies OK, the 64bit integer after the whitespace will be the buffer that should be used. The server will never return a buffer’s size greater than the one requested by the client.

After this, the server starts accepting the data flowing from the client. It works as follow:

The client must send a header before every payload. This header contains how many bytes will be sent next. The header is a 64bit signed integer normalized -converted to network byte-order.

This header is a window that allows the client to make some operations during the flow. The client can send the following codes using the header:

0 This is the HPFEND code which means that there is no more data to be sent.

-1 This is the HPFSTOP code which means that the client wants to stop sending data and will continue later. In this case the server just stops writing the file without removing it.

-2 This is the HPFCANCEL code which means that the client is canceling the operation. The server stops the operation and deletes the file.

-127 This is the HPFFIN code and is used to tell the server that the operation has finished.

No matter which code is used, the operation always must be ended by the HPFFIN code. Both sides must send it. First the server, then the client.

After the server receives one of the above codes -except HPFFIN-, it sends HPFFIN code, then the client responds with another HPFFIN code. Once this termination handshake is done, the server returns to listen again for new commands.

About cancellation:

This command is an asynchronous one in the sense that either side can send an HPFCANCEL code in the middle of the flow. As a result, the server can send, while receiving data, any code to request cooperation from the client. The server can send the following codes:

HPFCANCEL

Under certain conditions the server can decide to cancel the operation and will request the client to do so. After the server sends the HPFCANCEL code, all data received in the server will be discarded. The file will be removed too.

Once the HPFCANCEL code reaches the client, it must stop sending data. After the server sends this code, as stated before, it will send the HPFFIN code which indicates that the operation is done. Then the client responds with another HPFFIN code to confirm the end of the operation.

If the client sends, at any point, HPFSTOP, it can be safely assumed that the amount of data sent to the server is already stored in the file.

The above assumption does not apply when after the client sends HPFSTOP it receives from the server an HPFCANCEL. In this case the server deletes the file.

Note of caution:

As the server can cancel the operation asynchronously, the client may send some data before realizing the cancellation.

[Supports TFPSFS]

GET

```
struct hpfile {  
    uint64_t offst;  
    int64_t bufsz;  
};
```

GET “/path/to/filename” hpfile

GET is part of the High Performance File Interface. It can be used to transfer files from the server to the client.

The first parameter is the file to be read.

The second parameter goes after the “/path/to/filename” and a whitespace. It consists of two 64bit integers normalized ‘converted to network byte-order’. The first one should be unsigned and the second one, signed. The first one must range from 0 to $2^{64}-1$. The second one must range from $-(2^{63})$ to $2^{63}-1$.

The “offst” integer -the first one- is the desired offset in which the data should start to be read in the opened file.

The “bufsz” integer -the second one- is the proposed buffer size by the client.

Once the command is constructed like the above, it can be sent to the server. On the server side, the protocol will evaluate the parameters and will return one of the following conditions:

OK “definitive-buffer”

FAILED 1 : Access denied to location.

FAILED 16 : Missing parameter from command.

FAILED 34 : H-P interface failed to open file descriptor.

FAILED 35 : H-P interface failed to set file position.

FAILED 36 : H-P interface failed to allocate the buffer.

“definitive-buffer” is a 64bit signed integer normalized -converted to network byte-order- with the buffer that the server finally decided to

use. The server tries to create a buffer with the size indicated by the client, if that request cannot be satisfied, the server will create a buffer according to implementation-dependent criteria. In any case, if the server replies OK, the 64bit integer after the whitespace will be the buffer that should be used. The server will never return a buffer's size greater than the one requested by the client.

After this, the server starts sending data to the client. It works as follows:

The server sends a header before every payload. This header contains how many bytes will be sent next. The header is a 64bit signed integer normalized -converted to network byte-order.

This header is a window that allows the server to do some operations during the flow. The server can send the following codes using the header:

0 This is the HPFEND code which means that there is no more data to be sent.

-2 This is the HPFCANCEL code which means that the server is canceling the operation.

-127 This is the HPFFIN code and is used to tell the client that the operation has finished.

No matter which code is used, the operation must always end with the HPFFIN code. Both sides must send it. Unlike the PUT command, first the client, then the server.

After the server sends one of the above codes -except HPFFIN-, it waits for HPFFIN code, then it responds with another HPFFIN code. Once this termination handshake is done, the server returns to listen again for new commands.

About cancellation:

This command is an asynchronous one in the sense that either side can send a HPFCANCEL code in the middle of the flow. As a result, the client can send, while receiving data, any code to request cooperation from the server. The client can send the following codes:

HPFCANCEL
HPFFIN

Under certain conditions the client can decide to cancel the operation and will request the server to do so. After the client sends the HPFCANCEL code, all data received by the client should be discarded.

Once the HPFCANCEL code reaches the server, it will stop sending data. After the server receives this code, as stated before, it will wait for the HPFFIN code which indicates that the operation is done. Then the server responds with another HPFFIN code to confirm the end of the operation.

Note of caution:

As the client can cancel the operation asynchronously, the server may send some data before realizing the cancellation.

[Supports TFPSFS]
PUTCAN


```
struct hpfile {  
    uint64_t offst;  
    int64_t bufsz;  
    uint64_t canpt;  
};
```

PUTCAN “/path/to/filename” hpfile

PUTCAN is part of the High Performance File Interface. It can be used to transfer files from the client to the server. It differs from PUT because the cancellation, rather than asynchronous, are predefined points.

The first parameter is the file to be written, it could exist or not. If the file does not already exist, then it is created. If the file already exists, the command will overwrite it.

The second parameter goes after the “/path/to/filename” and a whitespace. It consists of three 64bit integers normalized ‘converted to network byte-order’. The first one should be unsigned; the second one, signed; and the third one, unsigned. The first one must range from 0 to $2^{64}-1$. The second one must range from $-(2^{63})$ to $2^{63}-1$. The third one must range from 0 to $2^{64}-1$.

The “offst” integer -the first one- is the desire offset in which the data should be written in the opened file, being a new file or an existing one.

The “bufsz” integer -the second one- is the proposed buffer size by the client.

The “canpt” -the third one- is the number of transmitted buffers in which there will be a window to cancel the operation. 0 means no cancellation window.

Once the command is constructed like above, it can be sent to the server. On the server side, the protocol will evaluate the parameters and will return one of the following conditions:

OK “definitive-buffer”

FAILED 1 : Access denied to location.

FAILED 16 : Missing parameter from command.

FAILED 34 : H-P interface failed to open file descriptor.

FAILED 35 : H-P interface failed to set file position.

FAILED 36 : H-P interface failed to allocate the buffer.

“definitive-buffer” is a 64bit signed integer normalized -converted to network byte-order- with the buffer that the server finally decided to use. The server tries to create a buffer with the size indicated by the client. If that request cannot be satisfied, the server will create a buffer according to implementation-dependent criteria. In any case, if the server replies OK, the 64bit integer after the whitespace will be the buffer that should be used. The server will never return a buffer’s size greater than the one requested by the client.

After this, the server starts accepting the data flowing from the client. It works as follow:

The client must send a header before every payload. This header contains how many bytes will be sent next. The header is a 64bit signed integer normalized -converted to network byte-order.

This header is a window that allows the client to make some operations during the flow. The client can send the following codes using the header:

0 This is the HPFEND code which means that there is no more data to be sent.

-1 This is the HPFSTOP code which means that the client wants to stop sending data and will continue later. In this case the server just stops writing the file without removing it.

-2 This is the HPFCANCEL code which means that the client is canceling the operation. The server stops the operation and deletes the file.

About cancellation:

This command is intended for scenarios where multi-threading is not available, so asynchronous cancellation is not possible. In such a case, the client and the server must agree after how many bytes sent, there will be a chance to cancel the operation.

The “canpt” parameter specifies the number of buffers that will be sent before switching the way the data is flowing. Once the flow is inverted, the server has the chance to send one of the following codes:

HPFCANCEL

-3 HPFCONT (This code tells the other end that flow can continue).

[Supports TFPSFS]

GETCAN

```
struct hpfile {  
    uint64_t offst;  
    int64_t bufsz;  
    uint64_t canpt;  
};
```

GETCAN “/path/to/filename” hpfile

GETCAN is part of the High Performance File Interface. It can be used to transfer files from the server to the client. It differs from PUT because the cancellation, rather than asynchronous, are predefined points.

The first parameter is the file to be read.

The second parameter goes after the “/path/to/filename” and a whitespace. It consists of three 64bit integers normalized ‘converted to network byte-order’. The first one should be unsigned; the second one, signed; and the third one, unsigned. The first one must range from 0 to $2^{64}-1$. The second one must range from $-(2^{63})$ to $2^{63}-1$. The third one must range from 0 to $2^{64}-1$.

The “offst” integer -the first one- is the desired offset in which the data should start to be read in the opened file.

The “bufsz” integer -the second one- is the proposed buffer size by the client.

The “canpt” -the third one- is the number of transmitted buffers in which there will be a window to cancel the operation. 0 means no cancellation window.

Once the command is constructed like the above, it can be sent to the server. On the server side, the protocol will evaluate the parameters and will return one of the following conditions:

OK “definitive-buffer”

FAILED 1 : Access denied to location.

FAILED 16 : Missing parameter from command.

FAILED 34 : H-P interface failed to open file descriptor.

FAILED 35 : H-P interface failed to set file position.

FAILED 36 : H-P interface failed to allocate the buffer.

“definitive-buffer” is a 64bit signed integer normalized -converted to network byte-order- with the buffer that the server finally decided to use. The server tries to create a buffer with the size indicated by the client, if that request cannot be satisfied, the server will create a buffer according to implementation-dependent criteria. In any case, if the server replies OK, the 64bit integer after the whitespace will be the buffer that should be used. The server will never return a buffer’s size greater than the one requested by the client.

After this, the server starts sending data to the client. It works as follows:

The server sends a header before every payload. This header contains how many bytes will be sent next. The header is a 64bit signed integer normalized -converted to network byte-order.

This header is a window that allows the server to do some operations during the flow. The server can send the following codes using the header:

0 This is the HPFEND code which means that there is no more data to be sent.

-2 This is the HPFCANCEL code which means that the server is canceling the operation.

About cancellation:

This command is intended for scenarios where multi-threading is not available, so asynchronous cancellation is not possible. In such a case, the client and the server must agree after how many bytes sent, there will be a chance to cancel the operation.

The “canpt” parameter specifies the number of buffers that will be sent before switching the way the data is flowing. Once the flow is inverted, the client has the chance to send one of the following codes:

HPFCANCEL

-3 HPFCONT (This code tells the other end that flow can continue).

LOGIN ‘user’ ‘password’

LOGIN evaluates the combination of ‘user’ -as first parameter- and ‘password’ -as second parameter- against a database in the server. Note that there is a whitespace between the first and the second parameter. If a match is found, then the protocol sets the real-user id and real-group id of the process to that user. From this point on, all executed commands at the server are done on behalf of that logged user and the access to the file system will be restricted to the permission of that user. The return status of this command could be:

OK

FAILED 39 : The process is already logged.

FAILED 16 : Missing parameter from command.

FAILED 40 : Failed to open users database.

FAILED 37 : Login failed.

[Supports TFPFS]

CHMOD “/path/to/file” “octal-mode”

CHMOD changes the file access permission in the same fashion that the UNIX system call does it. The first parameter is the file or directory in which the access bits are about to be changed. The second parameter is the mode ‘permission’ in octal notation to be used. Between the first and the second parameter there is a whitespace.

The complete explanation of composing the mode is out of the scope of this document. But here are a few useful examples:

CHMOD “path-to-file” 0400

Only owner can read the file.

CHMOD “path-to-file” 0200

Only owner can write the file.

CHMOD “path-to-file” 0100

Only owner can execute the file.

Now if we do summation we can obtain the following combinations:

CHMOD “path-to-file” 0600

Only owner can read and write the file.

CHMOD “path-to-file” 0500

Only owner can read and execute the file.

CHMOD “path-to-file” 0300

Only owner can write and execute the file.

So far, we have seen the second digit -from left to right- which represents the owner’s permission. The third digit represents the group’s permission. The fourth digit represents the permission for ‘others’ (anybody else). The same summation logic explained before for the second digit applies to the third and the fourth.

About the first digit, it could be:

4 (SUID) Set user-id bit.

2 (SGID) Set group-id bit.

1 Sticky bit.

Here the summation only applies to SGID (2) and Sticky bit (1). So, the value for the first digit could be: 4, 2, 1, or 3.

For the CHMOD command the first digit -if not needed- could be 0 or it can be completely omitted. For more information see the UNIX man pages.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 9 : File does not exist.

FAILED 41 : Failed to change file mode.

[Supports TFPFS]

CHOWN “/path/to/file” “user” “group”

CHOWN changes the ownership of a file or a directory. This command in some systems could be only executed by a privileged user (root). In other systems it can be executed too by the owner of the file or directory. The successful execution will depend of the logged user.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 42 : System user does not exist.

FAILED 43 : System group does not exist.

FAILED 44 : Failed to change file ownership.

[Supports TFPSFS]

SHA256 'path'

SHA256 makes a sha256 hash of a file indicated by 'path'. This command is intended to apply a hash to a file before downloading it in order to guarantee its integrity. Be aware that on very large files the time to resolve the digest function could be potentially long. The return status of this command could be:

OK "hash-in-hex-form like 0xff33..."

FAILED 45 : Failed to make SHA256 hash.

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

NIGMA 'keylen'

NIGMA generate a random session key of the specified length in the first parameter. This length should be a number equal or greater than 8 and multiple of 4. If the command fails it return FAILED, otherwise OK. In case of success, after the OK, the server will send a 32bit integer converted to network byte-order indicating the payload size that coming next. This payload will be the new session key and the header must be equal to the keylen indicated by the first parameter. After the client receives the key it should be swapped with the old key, and the new key is the one should be used hereinafter to encrypt/decrypt the data. The return status of this command could be:

OK

-After OK the client must read a 32bit integer header and the session key that coming next, then should be swapped with the old session key-

FAILED 46 : Failed to swap the session key.

[Supports TFPSFS]

RMSECDIR “secure_token” | “/path/to/secure/dir/to/remove”

RMSECDIR removes a specified secure directory recursively. The first parameter is the secure token that allows to remove the directory specified in the second parameter. The secure token could be either a file or a directory named as the first parameter, inside the the directory specified as the second parameter. Be aware that any directory deleted by this command will be considered secure, either it ends in .sd or not. The return status of this command could be:

OK

FAILED 1 : Access denied to location.

FAILED 30 : Directory is locked.

FAILED 15 : Directory to remove still exist.
FAILED 47 : Failed removing secure directory.

If the second parameter is leaved in blank the protocol daemon directory is used. So it returns the FAILED 15 return status due the impossibility to delete the daemon directory. However, every other directory will be deleted is “secure_token” is found.

INJAIL “secure_token” | “/path/to/dir/to/in-jail/tfprotocol/”

INJAIL in-jails the TFProtocol damon in the directory specified by the second parameter. This is achieved only if “secure_token”, specified as first parameter, is permitted for that directory. The return status of this command could be:

OK

FAILED 1 : Access denied to location.
FAILED 16 : Missing parameter from command.
FAILED 48 : Failed to open ACL file in jail directory.
FAILED 49 : Invalid security token in jail directory.
FAILED 51 : The process is already in-jailed.
FAILED 50 : Invalid command before in-jail.

TLB

TLB requests a tuple 2 ip/port from the Transfer Load Balancer pool. The clients can use this command -in ‘advisory’ way- to retrieve other servers in order to balance the overall traffic. By ‘advisory’ we mean that it’s up to the clients to agree in asking for a server from the pool before start any further interaction. The return status of this command could be:

OK “4|6 ip;port”

an example for IPv4 could be:

4 1.1.1.1;1234

an example for IPv6 could be:

6 ::1;1234

FAILED 52 : Failed to retrieve TLB tuple.

At this point, again, it is up to the client to disconnect from the server and connect to the retrieved one. It is guarantee that the hash and public key will remain the same.

[Supports TFPSFS]

SDOWN “path/to/file/to/download”

SDOWN downloads the specified file in the command argument. Before every payload, a 32bit signed integer header converted to big-endian with the payload size is sent by the server. If there is an error a header with value -1 is sent. If the end of file is reached a header with value 0 is sent.

[Supports TFPSFS]

SUP “path/to/store/uploaded/file”

SUP uploads the specified file in the command argument. Before every payload, a 32bit signed integer header converted to big-endian with the payload size must be sent by the client. If there is an error a header with value -1 must be sent. If the end of file is reached a header with value 0 must be sent. If the file already exist in the server, it will be truncated to zero. If the client sent -1 indicating the

upload canceling, the uploaded data is deleted. If the client sent 0 indicating that there is no more data to send -EOF-, then it has to wait for a header -32bit big-endian- from the server indicating the result of the operation. 0 means OK, -1 means failed. If -1 is sent by the server, the uploaded data is deleted.

[Supports TFPSFS]

FSIZE “path/to/file”

FSIZE gets the file size -in bytes- of the specified file in the parameter. This command returns a header of 64bit signed integer converted to big-endian with the file size or an error code. The return status of this command could be:

File size in bytes (i.e 5403)

-1: indicates a failure executing the command.

-2: The indicated file is not a regular file. It could be a directory or any other special file.

[Supports TFPSFS]

FSIZELS “path/to/file/with/paths”

Each line in the specified file as parameter represents a path to a file. FSIZELS gets the file size -in bytes- of each path. This command returns headers of 64bit signed integer converted to big-endian with the file sizes or error codes. For each line a header is returned with one of the following results:

File size in bytes (i.e 5403)

-1: indicates a failure executing the command for that line.

-2: The line is not a regular file. It could be a directory or any other special file.

If there are no more lines in the file, a header with value -3 is returned indicating it. This command always ends by sending -3.

[Supports TFPSFS]

LSV2 “path/to/list”@||@”path/to/file/to/store/the/list”

LSV2 lists the directory indicated by the first parameter. It stores all listed files in the file indicated by the second parameter, one file per line. If a line ends with slash ‘/’ it means is a directory, otherwise it could be any other file. By any other file we mean a regular file, a socket, a character device, and so on. The @||@ token separates both paths. The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 20 : Source path is not a directory.

FAILED 1 : Access denied to location.

FAILED 30 : Directory is locked.

FAILED 53 : Failed running LSRV2.

The paths are absolute -”path/to/list” is included-.

[Supports TFPSFS]

LSRV2

LSRV2 does the same that LSV2 but it does it recursively. The return status are the same.

[Supports TFPSFS]

FTYPE “path/to/file/to/get/type”

FTYPE returns a byte indicating the type of the file that the first parameter points to. Once the command is sent to the server, the client must wait for a single byte containing one of the following results:

- 0 for a directory
- 1 for a character device
- 2 for a block device
- 3 for a regular file
- 4 for a FIFO or a pipe
- 5 for a symbolic link
- 6 for a socket
- 7 for any other type
- 1 for a failure getting the type

[Supports TFPSFS]

FTYPELS “path/to/file/containing/the/paths/to/files/to/get/types”

FTYPELS works pretty much like FTYPE except it return a byte with the type per each line in the file indicated by the first parameter. Each line in the file passed as first parameter must be a path. The return values are the same as in FTYPE, except it sends -2 when there are no more types to receive.

[Supports TFPSFS]

FSTATLS “path/to/file/containing/the/paths/to/files/to/get/stats”

FSTATLS returns a special structure per each line in the file indicated by the first parameter. Each line in that file must be a path. The structure is exactly 26 bytes and looks like the bellow:

```
struct fstathdr {  
    char code;  
    char type;  
    uint64_t size;  
    uint64_t atime;  
    uint64_t mtime;  
};
```

As far as the client concerns it receives 26 bytes. The first byte -code member- contains one of the following values:

0 It means OK and there is relevant info in the rest of the members.

-1 A failure has occurred getting the statistics for that path. In this case there is not any relevant information in the rest of the members.

-2 It means that there is no more statistics structures coming. In this case there is not any relevant information in the rest of the members.

The second byte –the type member- contains the same possible values of the FTYPE command.

The next 8 bytes -the size member- is a 64bit unsigned integer converted to big-endian that contains the size of the file, directory, etc, indicated by the path.

The next 8 bytes -the atime member- is a 64bit unsigned integer converted to big-endian with the timestamp -Unix time in seconds- of the last access time.

The next 8 bytes -the mtime member- is a 64bit unsigned integer converted to big-endian with the timestamp -Unix time in seconds- of the last modification time.

[Supports TFPSFS]

INTREAD “path/to/file/”

INTREAD -Integrity Read-. It is intended to atomically download a file with its integrity checksum, for the case SHA256. It will return to the client a 32bit integer -big-endian- header with -1 to indicate an error or the file size even if is 0. After the server sent the header it will send the SHA256 checksum which it is exactly 66 bytes: 0x plus the hash formatted in hexadecimal format. After this, the server will send the payload. Unlike many commands, the entire file will be loaded in the server process space address, so it is not intended to manage vary large file.

[Supports TFPSFS]

INTWRITE “path/to/file/”

INTWRITE -Integrity Write-. It is intended to atomically upload a file with the integrity checksum -SHA256- of the file indicated in the parameter, so the destination file must exist. The server will expect the hash, after receive it, which is exactly 66 bytes: 0x plus the hash formatted in hexadecimal format, the server will expect for a 32bit integer -big-endian- header with the file size to be uploaded. After the this header is received, the client must send the payload. Once the file and the hash are at the server side, the server will compare that hash with the hash of the file indicated in the parameter, if both hashes are equal the server returns 0, otherwise -1, in a 32bit -big-endian- integer header. The main reason for the server to return -1 is that the hashes do not match, but it could be by many other

situations. In any case, a return -1 must be considered as a hash match failure. If the server returns 0, truncates the destination file to 0 and write the file with the payload received. Unlike many commands, the entire file will be loaded in the server process space address, so it is not intended to manage vary large file.

NETLOCK “timeout in seconds” “/path/to/file/to/lock”

NETLOCK creates an advisory record lock in the file specified by the second parameter with a timeout in seconds indicated in the first parameter. The system guarantee that each time the network detects activity, the timeout will be reset. By activity we meant any read or write operation from/to the network. However, attention must be pay specially when the lock is set-up and TFProtocol enters a subsystem. It is possible that the subsystem remains silent for long periods of times and the timeout expires. The client is responsible for send data periodically to avoid the timeout expiration. If the timeout expires and the client does not called NETUNLOCK, TFProtocol will close the connection releasing the lock on the file. The return status of this command could be:

OK “lock-id”

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 54 : Failed to acquire or release NETLOCK.

If the command succeeded, the “lock-id” -without double quotes- is an integer index that represents the lock.

NETLOCK_TRY “timeout in seconds” “/path/to/file/to/lock”

NETLOCK_TRY works exactly like NETLOCK but instead of blocking until obtain the lock, it will return the error FAILED 54.

NETUNLOCK “lock-id”

NETUNLOCK releases the lock, if were in place, on the file represented by “lock-id” -without double quotes-. The return status of this command could be:

OK

FAILED 54 : Failed to acquire or release NETLOCK.

[Supports TFPSFS]

NETMUTACQ_TRY “/path/to/write/the/mutex” “token”

NETMUTACQ_TRY tries to acquire the ownership of a synchronization object to enter a critical section. It work as follows: If the file indicated by the first parameter does not exist, the command will fail. If the file exist TFProtocol will acquire an advisory exclusive record lock on to it. If the file is zero bytes, then “token” -without double quotes- will be written in it. The mutex is acquired and OK will be returned to the client. If the file has something written that match exactly with “token”, then the mutex is considered as acquired too and OK will be returned to the client. If the file has something written that do not match with “token”, then the mutex is considered as non-acquired and a failure will be returned to the client. At this point the exclusive advisory record lock will be release on the file. Be aware that this kind of mutual exclusive synchronization object is persistent and will survive any

network disconnection. Who acquires the mutex -"token"- must release it explicitly by calling NETMUTREL command, otherwise the mutex will be considered as acquire forever. The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 55 : Failed to acquire or release NETMUT.

Summary:

The mutex -"token"- is considered acquire if the file indicated by the first parameter either has 0 bytes or contains exactly "token" -without double quotes-. Otherwise, the mutex is considered as non-acquire.

[Supports TFPSFS]

NETMUTREL "/path/to/write/the/mutex" "token"

NETMUTREL releases the ownership of a mutex "token" previously acquired by NETMUTACQ_TRY. The first parameter indicates the file where the mutex was placed. The "token" -without double quotes- parameter tells TFProtocol what mutex has to be released. It works as follows: An exclusive advisory record lock is acquired on the file. If the file does not exist, it will be returned a failure to the client. If the file exist and it has 0 bytes or it contains exactly "token", the file is truncated to 0 and OK is returned to the client. At this point the mutex is considered released. The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 1 : Access denied to location.

FAILED 55 : Failed to acquire or release NETMUT.

Summary:

The mutex is considered released only if the file that contain it has 0 bytes or has written exactly “token”, in the later case TFProtocol will truncate it to 0.

SETFSID “secureFileSystemIdentity”

SETFSID sets the process identity that will be used in any operations that involves the TFProtocol Secure Filesystem. The return status of this command could be:

OK

SETFSPERM “secID:128” “path/to/securefs/directory”

SETFSPERM sets the permission mask indicated after ‘:’ for the identity that precede it. If is the first time that a permission is set for the directory the “secID” becomes the owner with the permissions indicated after ‘:’. This means that only secID -secure filesystem identity- will be able to sets new permissions for others “secIDs”. If the directory has already permissions set, the process’s identity set by SETFSID must have the right permissions to call SETFSPERM. For further details see TFProtocol Secure Filesystem section.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 57 : Failed setting secure FS permission.

FAILED 1 : Access denied to location.

REMFSPERM “secID” “path/to/securefs/directory”

REMFSPERM removes the secID indicated by the first parameter from the secure filesystem directory indicated by the second parameter. The process’s identity set by SETFSID must have the right permissions to call REMFSPERM. For further details see TFProtocol Secure Filesystem section.

The return status of this command could be:

OK

FAILED 16 : Missing parameter from command.

FAILED 58 : Failed removing secure FS permission.

FAILED 1 : Access denied to location.

GETFSPERM “secID” “path/to/securefs/directory”

GETFSPERM gets the secID’s permissions of the specified directory in the second parameter.

The return status of this command could be:

OK “permission-mask”

FAILED 16 : Missing parameter from command.

FAILED 59 : Failed getting secure FS permission identity.

FAILED 1 : Access denied to location.

ISSECFS “path/to/directory”

ISSECFS gets if “path/to/directory” belongs to the TFProtocol Secure Filesystem domain.

The return status of this command could be:

OK

FAILED 60 : The directory does not belongs to secure FS.

FAILED 20 : Source path is not a directory.

FAILED 1 : Access denied to location.

LOCKSYS “path/to/directory/to/in/lock”

LOCKSYS locks TFProtocol in the specified directory. Once the daemon is locked in a directory, there isn't way back. This command it's not intended to be called multiple times, instead, if TFProtocol is configured to in-lock processes this command should be called before any other, except: END; LOGIN; KEEPALIVE; PROCKEY; INJAIL.

OK

FAILED 9 : File does not exist.

FAILED 10 : Directory does not exist.

FAILED 61 : Unable to use Folder Locking System.

GETTZ

GETTZ returns TFProtocol instance current timezone. The return status of this command could be:

OK “time-zone”

SETTZ “time-zone”

SETTZ sets TFProtocol current instance timezone to “time-zone”. The format may vary from system to system but at least it supports POSIX.1 format which looks like the next examples:

EST+5EDT,M3.2.0/2,M11.1.0/2
WART4WARST,J1/0,J365/25

It is likely, and quite extended, to support it in a more user-friendly format:

America/New_York

The return status of this command could be:

OK

LOCALTIME

LOCALTIME gets TFProtocol instance localtime using the current timezone. It returns the date in a human readable format. The return status of this command could be:

OK “2023-06-07 09:47:07 EST”
FAILED 26 : Date is not representable.

DATEFTZ “time-zone”

DATEFTZ gets datetime in a human readable format using the timezone specified in the first parameter. The return status of this command could be:

OK “2023-06-07 09:47:07 EST”

FAILED 26 : Date is not representable.

[Supports TFPSFS]

LSV2DOWN “path/to/list”

LSV2DOWN lists the directory indicated by the first parameter. Instead of store all the listed files in another file like LSV2, it send the list to the client directly. One file per line. If a line ends with slash ‘/’ it means is a directory, otherwise it could be any other file. By any other file we mean a regular file, a socket, a character device, and so on. The return status of this command could be:

OK

FAILED 20 : Source path is not a directory.

FAILED 1 : Access denied to location.

FAILED 30 : Directory is locked.

FAILED 53 : Failed running LSRV2.

** This error is used for both LSRV2 and LSRV2DOWN **

OK means that security checks was successfully passed.

Unlike LSV2, if the return status is OK, what coming next is the list of files. For each file the client will receive: a header with the line size and then the line itself which is the file path. The header is a 32-bit signed integer converted to big-endian. Finally, the command will

return to the client as a header value -10 or -11 indicating the end of data transmission. -10 Means success in directory listing and download. If -11 is sent, it means that an error was occurred while listing the directory, however the download will occur with the partial data obtained in the directory listing.

[Supports TFPSFS]

LSRV2DOWN

LSRV2DOWN does the same that LSV2DOWN but it does it recursively. The return status are the same.

[Supports TFPSFS]

RMKDIR “path/to/new/dir”

RMKDIR command creates a directory recursively at the specified path. The return status of this command could be:

OK

FAILED 1 : Access denied to location.

FAILED 30 : Directory is locked.

FAILED 63 : Failed to create directory recursively.

This command could partially success, in such a case it returns FAILED 63 and the created directories will not be deleted.

RUNBASH “*path/to/script.sh*”

RUNBASH if is enabled in the TFProtocol daemon configuration file, it allow to execute an arbitrary bash script. It is required that the bash shell path is set in the environment of the effective user-id of TFProtocol.

**** This is a super-sensitive issue, if you decide to enable it. Once the script is executed the TFProtocol daemon has no control at all over the new program image executed -a.k.a the script-.**

Once the command is executed and returns OK, that does not means that the script was successfully executed or even run it. As far has TFProtocol invocation once the execlp system call is invoked, it has no other thing to do but return OK.

**** If this feature is going to be enabled on some server, it is important to know in advance that only trusted apps will access that server.**

One really bad thing that could happen is describe bellow:

```
!/bin/bash
```

```
rm -Rf /
```

```
exit 0
```

```
SUP /badscript.sh
```

```
RUNBASH /badscript.sh
```

As you can see, you must know all the security concerns that you may face if you decide to enable this feature. The return status of this command could be:

OK

FAILED 64 : Failed running RUNBASH command.

FAILED 9 : File does not exist.

Also check README.rst to see how to enable this feature.

FLYCONTEXT “/path/to/directory/to/create-and-injail”

FLYCONTEXT tries to create recursively the specified directory and injail the daemon there. You may call as many time as you wish this command and subsequently will continue to injail you deeper and deeper. Unlike INJAIL command you don't have to enable it on the configuration file of the daemon. However, if you do so, as soon as you connect, and probably after LOING command, if is the case, you must call FLYCONTEXT. After that you may use the protocol as usual an unlink to INJAIL you may continue calling FLAYCONTEXT. The return status of this command could be:

OK

FAILED 1 : Access denied to location.

FAILED 65 : Unable to use the FlyContext System.

****** For enable the enforcement of this feature in the configuration file of the TFProtocol daemon, please check out README.rst

GOAES

GOAES changes the encryption algorithm from the TFProtocol standard cryptography to EAS -Ran-. Once the command is sent, a 256 bits key should be sent, after that, the initialization vector which is 128 bits must be sent. The return status of this command could be:

UNKNOWN

OK

FAILED 66 : Unable to start AES encryption system.

If UNKNOWN is returned means that AES is not implemented in the server.

Notice. This three possible response are made yet with the TFProtocol standard cipher.

If the return is OK, after that, AES is in place and the rest of communication must be continued with that cipher.

There is some cares that must be put in place: As TFProtocol uses by default a stream cipher and AES is a block cipher some buffer translation must take place in order to transform some arbitrary size into a multiple of AES_BLOCK_SIZE. Once this translation is done and the cipher text is in this aligned buffer, can be sent to the server.

It is In order to get the most small possible buffer, but aligned to AES_BLOCK_SIZE use this formula because is the one used by the server. This way both sides knows in advance how much data expect from the other side without need any extra header.

$$\text{buffer_size} = \text{arbitrary_size} \leq \text{AES_BLOCK_SIZE} ?$$
$$\text{AES_BLOCK_SIZE} : \text{arbitrary_size} + \text{AES_BLOCK_SIZE} -$$
$$\text{arbitrary_size} \% \text{AES_BLOCK_SIZE}$$

The above should be exactly the size of the buffer to transmitted with the ciphered data.

As with the TFProtocol cipher, two copies of **key** an **initialization vector** must be used. One for transmitting data and the other for receiving data. This is required due to TFProtocol communication

layer must support multi-thread -a.k.a full-duplex- in order to enable actual and future subsystems to be multi-threaded.

TFPCRYPTO

TFPCRYPTO goes back to the TFProtocol standard stream cipher. The return status of this command could be:

OK

This response is still made in AES. After that, the communication must continue with The TFProtocol cipher. The symmetric keys must be the same left at the point GOAES was called.

GENUUID

GENUUID generates an Universally Unique Identifier and send it back to the client in text format. The return status of this command could be:

OK 9570a411-7512-422b-a260-2f608aa5ef91

CIAON The CIAON command enables a protocol-level feature designed to prevent client impersonation by malicious actors. Upon execution, the server generates and returns a **UUID** exactly **37 bytes** long, which serves as a temporary identity token for the client. From that point onward, **the client must include this UUID with every payload sent to the network, before the payload.** This allows the server to verify that incoming messages originate from the legitimate client and not an impersonator.

The client may **regenerate the UUID at any time** by issuing the CIAON command again. The return status of this command could be:

OK 9570a411-7512-422b-a260-2f608aa5ef91

CIAOFF The CIAOFF command disables the client impersonation avoidance mechanism previously enabled by the CIAON command.

Once issued, the server will **no longer require the UUID** in payloads sent by the client, and any previously issued UUID will be considered **invalid**. This effectively removes the identity verification layer and allows payloads to be accepted **without UUID-based validation**.

Use of CIAOFF should be restricted to trusted or controlled environments, as it **removes protection against impersonation attacks**. The return status of this command could be:

OK

The Fast Access Interface

The Fast Access Interface, -FAI- for short, is an access interface to the TFProtocol server which avoid the usual connection sequence. The usual connection sequence implies the RSA 2048, or whatever stated above in the beginning of the this document, encryption of a symmetric key which will be used with AES or the default build-in symmetric encryption algorithm of TFProtocol for data transmission. This encrypted key is sent to the server and the communication may proceeded. In fact. There are few other steps, like sending to the server the protocol version and so on. By contrast, FAI, will directly jump into the main command interface of TFProtocol so the facilities may be used right away.

However, how this can be achieved without compromising security?

First of all, the very first connection that a client make to TFProtocol server must be done as always. Once the client is connected, it must execute the FAITOK command. This command returns to the client an UUID and a symmetric key that will be used the next time the client connects to the server. Now the communication may proceed as usual and commands can be executed. Once this session is closed, the client has an UUID, which is the identification token, and a symmetric key that will be used as the next session key to encrypt/decrypt with AES or with the default TFProtocol encryption algorithm.

The next time the client wants to connect to the server, in the TFProtocol version step must send the following syntax:

```
FAI://uuid_token
```


This makes TFProtocol bypass RSA decryption of the session key and jump directly into the command interface, after make some security verifications like the token expire time and its validity.

Once the TFProtocol server is at the main interface command, the previously shared symmetric key will be used to encrypt/decrypt data. This implies that the client must store somewhere the UUID and symmetric key obtained when FAITOK was called in the previous session.

One more thing should be said about FAI:

When a client connect using this interface like in FAI://uuid_token, the first thing the protocol will return will be OK or an error. After that a new symmetric key is sent to the client that should be stored for the next session.

An example of the flow is the following Sequence Model.

- 1-) Client connect as usual to TFProtocol.
- 2-) Client executes FAITOK.
- 3-) Client stores the UUID and the next session key.
- 4-) Client uses TFProtocol as usual.
- 5-) Client disconnect from TFProtocol.

This time the client will connect to the server using FAI.

- 1-) Client send in the protocol version the syntax: FAI://uuid_token.
- 2-) The server responses one of the following answers:
 - OK
 - FAILED 68 : Token expired for FAI access.
 - FAILED 69 : FAI key is invalid.

If an error is returned, then the communication terminates. If OK is returned then the sequence continues:

3-) The server using the symmetric key returned to the client in the previous session encrypts the next symmetric session key which is randomly generated and encode in base64 as with the command FAITOK and send it to the client. The UUID and expire time remains the same.

At point 2, if FAILED 68 or FAILED 69 is received, the client must connect to the TFProtocol server as usual and request a new UUID token with the FAITOK command.

At point 3, if a session key is returned then:

4-) The client stores the next session key.

5-) Client uses TFProtocol as usual.

6-) Client disconnect from TFProtocol.

The next command interface is used to control Fast Access Interface.

FAITOK “number-of-minutes”

FAITOK generates an Universally Unique Identifier for an amount of time specified in the first parameter and a random-size random key and send it back to the client in text and base64 formats respectively. The return status of this command could be:

OK timestamp_expire 9570a411-7512-422b-a260-2f608aa5ef91
ZwBERfW6/IfvX...

FAILED 67 : Failed initializing FAI interface.

Note that there is a white space between the UUID and the session key in base64 format, and between the timestamp and the UUID. The timestamp represents a point in the continuum of time in which the token will expire. In other words, it contains the number of seconds since EPOCH -Jan 1, 1970 00:00- in which the token will be no longer valid.

The UUID returned will be used in place of TFProtocol version in future connections. This makes the server daemon to check the validity of the UUID and jump directly to the command interface.

Use FAI://uuid_returend instead of the protocol version when connect the next time. It must be sent in plain text, this is, not encrypted.

The returned key should be used for the encryption mechanism when connect the next time.

FAIMQ

FAIMQ returns the maximum amount of time -or quanta- that the TFProtocol server will handover for a Fast Access Interface token. The return status of this command could be:

OK '500 or any other integer value in minutes'

The keepalive server's mechanism

The keepalive mechanism that the server provides allows a client to detect whether the server is still reachable; moreover, it allows to test if the socket can be used to transmit and/or receive actual data.

The TCP protocol is designed to allow long time disconnected links without breaking the state of the sockets at both ends. This means that when the line is up again, communication can continue.

However, there are some circumstances where fail-fast scenario is needed. That is, that any of the endpoints can realize that the other end it is not reachable, so communication cannot continue anymore. The aforementioned scenario is where the keepalive mechanism becomes relevant.

Not every platform supports the native TCP keepalive mechanism; in fact, it is optional in the RFC document. Reached this point, some sort of scheme need to be implemented in order to allow applications to detect dead peers. There is not a straightforward way to do this, especially if the underlying platform does not support it. Therefore, this is where application layer keepalive mechanism takes place.

The TF Protocol server must implement at least one of the following keepalive mechanism.

Out-of-Band data.

The Out-of-Band data –hereinafter the oob -, if present, works as follow:

The client sends an oob byte, then it waits for the respond. Once this byte reaches the server, another oob byte it is sent back to the client immediately.

With this dynamic exchange, on the client's side some timers should be set, and if they expire after a series of probes, the peer's dead can be assumed.

Of all possible sort of application layer keepalive mechanisms, this is the one that has less performance impact and it is very reliable in the sense that the byte is sent through the same socket without interfering with the normal data. This allows to detect if the socket is not transmitting anymore. Unfortunately, not all TCP implementations treats the oob in the same manner. As a result, some platforms do not support it.

The UDP_HOSTCHECK

This type of keepalive, if present, allows a client to detect if the server is still reachable. This does not imply the TCP connected socket is still able to send/receive data. It only implies that the server address/port can be reached. It works as follows:

The client sends and UDP packet to the same address/port that the TCP socket is connected. This packet contains only one byte and it is the decimal number 0.

Once the server receives the packet, it responds with another UDP packet containing only one byte, the decimal number 1.

If the replay packet that the server sends is received, it means that the server address/port is reachable. Once again, it does not imply that the connected TCP socket can still communicate.

The UDP_PROCHECK

This type of keepalive, if present, allows a client to test whether the server's instance that communicate with the TCP client's socket it is still running. This per-se does not guarantee that the TCP socket can communicate. There are a lot of conditions that produce scenarios where the instance is still running but cannot communicate; however, this is out of the scope of this document.

In order to get reliable feedback about the actual TCP socket, the server's help is needed through the **KEEPALIVE** command. This commands as explained in this document tells the server to send periodically TCP-layer keepalive through the opened socket. In case that all the probes fail, the default behavior is to close that server's instance.

Once that particular server's instance is closed, UDP_PROCHECK will 'see' that the communication can no longer continue and inform it back to the client. It works as follows:

The client requests the server's instance unique key with the **PROCKEY** command.

The client sets the server TCP-layer keepalives through the **KEEPALIVE** command.

Then it sends and UDP packet with the decimal number 1 as first byte and the retrieved key next to it.

As soon as this packet is received by the server, it checks whether that instance -identified by the key- is still running. In order to work properly the timeout and the number of probes of the UDP packets should be equal or slightly greater than the TCP-layer keepalives server setting.

Then the server creates an UDP packet with just one byte and send it back to the client.

The client can tell if the communication channel is still opened by reading the byte sent from the server. The possible values could be:

1 It means that the communication channel is still opened.

0 It means that the socket is no longer usable.

UDP SOCKCHECK

This type of keepalive, if present, allows a client reliably to check if the TCP socket is still usable, without the server's **KEEPALIVE** command help. However, some platforms may not be able of implement it. It works as follow:

The client sends an Out-of-Band byte (oob) through the TCP socket -this differs from the first explained mechanism because the oob byte responded by the server is not needed-. Once this byte reaches the server, a counter is incremented. This counter is a 32bit unsigned integer and starts from 0. The client, on its side, does the same. There is no problem if the integer overflows.

Then the client sends an UDP packet with the decimal value 2 as the first byte and the unique key retrieved from the server next to it. Just as in the **UDP_PROCHECK**. But this time there is a difference: the counter inside the server.

Once the server receives the UDP packet, it gets the value of the counter inside the server's instance that is connected to the client. Then the server creates an UDP packet that will be at least one byte. There are two possible versions of the packet:

0 It means that the socket is no longer usable.

1 "counter"

It should be noticed that there is no whitespace between the first byte which tells the return status and the counter -without the double quotes-.

Once the client has the value of the counter returned from the server it can compare it with the local copy. If the counter returned by the server is less than the local copy of the client, it means that the last oob byte was not received by the server, and can try again.

With any of the above keepalive methods, after a series of probes the client can assume that the TCP socket is no longer usable.

A note of caution: all of the above mechanisms could be implemented at the server side in different ways and the performance hit will vary from implementation to implementation and from platform to platform. Be aware that any of the keepalive stated above, even the ones with less overhead, have implicit trade-offs. It could be processing-cycles vs bandwidth-usage; fail-fast notice vs resource-consumption, and so on. In any case, they do not come for free, it always implies a performance cost.

UDP_TLB

This is the udp version of the **TLB** command. The main purpose of an udp version is to effectively reduce the overhead of TCP before retrieve a server ip/port tuple for communication. The downside of using this alternative is that communication it's not encrypted meaning that if somehow a malicious user manage itself to get in the middle of communication, it can derive communication to a malicious server by providing to the client a ip/port tuple of it own.

There is no way to ontologically avoid this. However there is a workaround that is worth mentioning. If the final server in which communication will actually occur have specific data that the malicious user cannot replicate, then this data can be verified before proceeding to meaningful data exchange. The other possibility is that really doesn't matter whether the server have been swapped or not – for a malicious one- due the non-confidentially nature of the data to be exchanged. This is how UDP_TLB works:

The client send one udp packet consisting only of 1 byte. This byte must be the decimal value 3. This tells the server that the client is requesting a new server to contribute to the network load balance. The server at this point will sent to the client one udp packet with one of the next responses:

```
1 TLB_TUPLE
0
```

The first byte indicates the success of the operation. 1 means OK; 0 means FAILED. If the response is OK, then is followed by a whitespace and the next to it the TLB_TUPLE. It is guarantee that the TLB_TUPLE format will complaint with the format returned by the TLB command of the main interface of TF Protocol.

Transparent Proxy System

The Transparent Proxy System, TRP now on, is a feature that the TFProtocol server may or may not implement. In fact, as far as it's concern to the user, it will no see any difference of using a proxyfied TFProtocol server from a one not proxyfied. It is entirely transparent from the client's perspective.

The goal of this system is to provide a way to hide the real server where the data processing occurs from anyone listening on the network. If a man-in-the-middle is listening on the wire, it will see only a connection to server A, while the real communications are derived inside of server A to server B. The main usage of this system is to secure sensible servers from cyberattacks. This has some pros and cons. Lets see some of them.

Pros of using a proxyfied TFProtocol sever:

-) Any attacker will not know the IP of the server where the important data resides.
-) A client on a IPv4-only network will communicate transparently with a an Ipv6 server -trough the TFProtocol proxy server- and the other way around. This requires that the proxy server has dual stack network, this is: IPv6 and IPv4 IPs.

Cons of using a proxyfied TFProtocol:

-) Due to the ephemeral port exhaustion, the theoretical limit for a

TFProtocol proxy server outbound connections is 65535, however the practical limit is much less than that. Although the IANA suggests a 16383 ports to be reserved as ephemeral ports - outbound connections-, this subject is OS-dependent. Moreover it is possible that the proxy server has some outbound connections of its own which reduces that number. This implies that maximum number of clients that can use the proxy server is really low. There is no way to escape from that due to the nature of the Transport Layer multiplexing/demultiplexing mechanism. However, there is a workaround: Each public IP that the server has will have its own ephemeral port space, so this is a way to scale-up the number of clients that the proxy server may attend.

-) The communication will be slightly slower than with an unproxified server.

Constrains:

Although the implementation of TRP it is no required, if present, it must meet the following conditions:

The majority -if not all- commands of TFProtocol and its extended subsystems must work transparently to the proxy. If this is not the case for some particular command or subsystem, it must be announced by the server implementation. This constrains apply too for the UDP services, this is, the UDP Keepalive mechanism and any other future feature.

An advise:

The IP adding to the proxy server to scale-up the practical number of simultaneous clients is recommend to be on the IPv6 address space.

This is because the IPv4 is a scarce resource. It will be far cheaper to get more IPv6s to increase the usage of a proxyfied TFProtocol server. This also requires that the final destination server is listening on an IPv6.

The notification system

The notification system is one of the modes in which the protocol works. This mode is invoked by issuing the STARTNTFY command. Once this is done there is no way to go back to the standard mode. In this mode is the client who wait for the server “notifications” instead of issuing command while server waits like in the standard mode. This method allows to avoid the polling every some interval to request the notification to the server with the inherit inefficiency of it.

The notification mechanism works by setting a string pattern and a directory for listening. Every some interval, defined by client side, the server searches for that pattern in the files of the directory associated. Such pattern must be the start part of name of the file to be considered a notification. Once the server finds it, it sends to the client the complete name of that file along with other information. Any directory can have associated severals notification, but the server will return only one of them. The order is undetermined, the only thing that the protocol guarantees is that if the client eliminates the received notification, the server will send the next one found, and so on until there is no one notification in the directory, in which case it sends no more data to the client.

In the same manner that a particular directory can have associated severals notification, the client can specify severals directory to listen.

In order to clarify:

By “notification” we mean a file which name starts with a string that match the specified pattern associated to that directory.

By “listen” we refer to the watchdog that server does in the specified directory each time the interval specified by the client side elapsed.

“Interval” must be taken as a contiguous part of the time continuum between to arbitrary points.

“Elapsed” must be taken as the length of an interval.

If the usage of the protocol requires both modes at the same time, the client must open two connections to the server at the same time, one for the standard mode to issue commands to the server; one to receive notification from the server.

ADDNTFY “token” “/path/to/directory/to/listen”

ADDNTFY sets a new notification which starts name is “token”, without the quotes, and the directory is specified in the second parameter. If “token” is specified by an empty string “” then all the files in the associated directory will be taken as notification regardless of its name. If the second parameter is specified by an empty string “” then the root directory of the protocol daemon is used. The return status of this command could be:

OK

FAILED 5 : Error adding new notification to listen.

FAILED 1 : Access denied to location.

Directories don’t apply as notification entities, so they will not be taken into account.

STARTNFY “interval”

STARTNTFY starts the watchdog to the list of directory specified by the ADDNTFY command. Once STARTNTFY is issued there is no way back to the standard mode. The “interval” parameter, if present, it is a decimal number expressed in seconds as 3.3 which means 3 seconds and 300 milliseconds. The resolution of the interval goes up to nanoseconds, so 3.000000001 is 3 seconds and 1 nanosecond. Of course the integral part of the interval could be 0 too, so 0.000000001 is 1 nanosecond and 0.0001 is 1 microsecond. This number can be expressed in scientific notation as in 1.0e-9. The precision of the interval parameter are 15 digits which is the “double” size specified by the IEEE 754, but the real processor resolution time to sleep could be greater than the requested. So do not assume that the interval specified will be respected. Even if the processor supports such time resolution, mostly operating systems today are preemptive, so the real time waited could be more than the one specified. Another thing to take into account is that the TF PROTOCOL does not specify how to implement the sleep routine for the watchdog, so in event of signals, hardware interruptions or events, the client can’t tell whether the sleeping routine will compute the unelapsed time and it will wait for it. In short, if the client usage of the protocol requires super-reliable time sleeping intervals, this isn’t your protocol.

If the “interval” parameter is left empty the server will assign one by default. How many time the server assigns by default to the watchdog mechanism is implementation dependent.

The return status of this command is:

N “file-that-math-notification-token”

The N is the index number of the directory-token list in which the notification has occurred and the string next to it “file-that...” is the complete name of the file.

The client must respond to this with one of two possible ways:

OK
DEL

OK tells the server to continue notifying without deleting the notification. In this case, when the server iterates again over the list could send the same notification for that directory even if there is more to notify. In any case, the client can't predict if the server will send a new one or an old one, for a particular directory.

DEL tells the server to delete the notification. This ensures that the next time the server iterates over the list it will send other notification than the previous one if there is more for a particular directory.

In the event that there is nothing to notify the server will continue listen in the directories without sending data to the client. The client in this case will continue waiting to receive notification from the server without consuming “quanta” for it.

By “quanta” we mean the time slice assigned by the operating system to a process in a time sharing the system.

It is up to the implementers of the client side of the protocol if there is going to be some protection to the interval specified for the watchdog. This is a special concern with really short intervals because it could lead the server to a quasi busy wait which leads to high consumption of processor time and a considerable drop of server's performance.

